

UCS503P - Software Engineering Project

MID-TERM REPORT

FoodLink

A Coordination Platform for Surplus Food Redistribution

Submitted by:

1024030909 Rishi Garg

1024030906 Aayushi Sud

1024031166 Himanshi Singla

1024030956 Yugam Heer

rgarg8.be24@thapar.edu asud.be24@thapar.edu

hsingla1.be24@thapar.edu yheer.be24@thapar.edu

Group: 3C64 BE Third Year, CSE

Submitted to:

Reaya Grewal

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.1.1	Problem Statement	5
1.1.2	Motivation	6
1.1.3	Existing System	6
1.1.4	Proposed System	6
1.1.5	Scope	7
1.2	Project Objectives	7
1.3	Organisation of the Report	8
2	Methodology and Design	9
2.1	System Architecture	9
2.1.1	High-Level Design	9
2.1.2	Technical Stack	9
2.1.3	System Requirements	10
2.1.4	Functional Requirements	10
2.1.5	Non-Functional Requirements	11
2.1.6	Database Design	11
2.1.7	API and Backend Design	11
2.1.8	Frontend Architecture	12
2.2	Implementation Details	12
2.2.1	Module 1: Core Functionality — Donation Lifecycle	12
2.2.2	Module 2: Core Functionality — Recipient Matching and Explainability	14
2.2.3	Module 3: Core Functionality — Authentication, Authorization and Trust	15
2.2.4	Module 4: Secondary Features — Standing Requirements	16
2.2.5	Module 5: Secondary Features — Administration and Platform Metrics	16
2.2.6	Module 6: Secondary Features — Role Portals and User Workflows	16
2.2.7	Security and Privacy Considerations	16
3	Results and Evaluation	17
3.1	Testing Methodology	17
3.2	Performance Metrics	17
3.3	Analysis	18
3.3.1	Current Limitations	18

4	Feasibility Report	21
4.1	Technical Feasibility	21
4.2	Operational Feasibility	22
4.3	Economic Feasibility	22
4.4	Schedule Feasibility	23
4.5	Legal, Security and Privacy Feasibility	23
4.6	Feasibility Summary	23
5	Conclusion	25
5.1	Summary of Work	25
5.2	Key Findings	25
5.3	Future Work and Recommendations	25
5.4	Final Remarks	26
6	System Design: UML Diagrams	27
6.1	Use Case Model	27
6.1.1	Use Case Diagram	27
6.1.2	Use Case Scenarios	27
6.2	Class Diagram	33
6.3	Activity Diagram	35
	Bibliography	37

Introduction

1.1 Background and Context

Institutional kitchens such as college messes, hostel canteens, bakeries and banquet halls routinely finish a service with edible food left over. At the same time, shelters and community kitchens nearby are short of meals. Food waste is a recognised problem at every scale [Uni24]. For a campus or a tier-2/3 city, however, the binding constraint is rarely supply. It is *coordination under a deadline*: cooked food is safe for only a few hours, and informal phone calls often outlast that window.

FoodLink is a web platform that brings every party to a handover into one tracked workflow. A **donor** posts surplus food. An administrator verifies each **recipient organisation** (an NGO or community kitchen) before it can receive food. A **volunteer courier** collects and delivers it, and a **platform administrator** maintains the platform. The project is developed for UCS503P (2026–27 odd semester), which requires a proposal in week 4, this prototype-stage (mid-term) report in week 7 and a final report in week 17 [Ven26]. The report describes what has been built against the proposal [GSHS26], how it was verified, and what remains.

Although the repository is named *FoodBridge-AI*, FoodLink contains **no machine learning or computer vision**. Organisations are ranked by a transparent weighted sum of five published criteria that can be recomputed by hand. This choice is recorded as decision D-05 in the project’s decision log [Foo26a]. All technical statements were checked against the source code at commit `e588b35` [Foo26b].

1.1.1 Problem Statement

Informal coordination causes five problems. The first is **perishability**: the hard deadline is routinely missed. The second is **fragmentation**: donors call whoever they know, so availability is invisible to others. The third is **lack of evidence**: nobody knows how long claiming took, whether food arrived, or how much was lost. The fourth is **the last mile**: courier availability is untracked. The fifth is **trust and privacy**: donors need assurance that a real organisation is receiving the food, and a home location should not be broadcast to anyone who signs up. The problem is therefore to *build a web platform that lets a donor post surplus food with a hard deadline, directs it to nearby verified organisations through an explainable ranking, coordinates a courier through pickup and delivery, and records every step as server-stamped evidence, while each participant sees and does only what its role and relationship to the donation permit.*

1.1.2 Motivation

Coordination, not supply, is the bottleneck. Shortening the delay between posting and acceptance converts waste into meals, which is why the proposal's primary metric is *time-to-claim* [GSHS26]. Trust must be earned: anyone can register as an organisation, so FoodLink separates holding a role from being trusted, and administrator verification gates who may receive food. Decisions should be explainable, and a published weighted sum lets organisations, donors and evaluators see *why* a ranking came out as it did, which an opaque model trained without outcome data would not. Finally, evidence should not be self-reported. Metrics are credible only if the server stamps their timestamps, so every transition is appended to a ledger.

1.1.3 Existing System

In the proposal's setting, redistribution relies on personal calls, messaging groups and ad-hoc arrangements, and no shared software system exists [GSHS26]. Surplus is therefore visible only to the contacts a donor happens to call. The recipient is whoever answers first rather than the best-placed organisation. Deadlines are mentioned but not enforced, and trust rests on familiarity rather than vetting. Couriers are arranged separately and not tracked, nothing is recorded after handover, and addresses are shared freely. The comparison is with informal practice in general, not with specific third-party products.

1.1.4 Proposed System

FoodLink is a three-tier web application: a React client, a REST API in Python (FastAPI), and a relational database (SQLite by default). The API is the only authority for every rule. A donation moves through the following flow: (1) a donor posts food details, a pickup pin and an absolute deadline; (2) the server stores the donation as **AVAILABLE**, ranks eligible organisations and, if any exist, records **MATCHED**, which is only a suggestion; (3) a verified organisation, shown its own match explanation, accepts it (**ACCEPTED**); (4) a courier, who until then sees only a coarse area of about 1 km, claims the pickup (**VOLUNTEER_ASSIGNED**) and only then sees the exact pin; (5) the courier records **PICKED_UP** and **DELIVERED**; (6) the organisation confirms receipt (**COMPLETED**), which updates the reliability used in future rankings. A donor (before pickup) or an administrator may also cancel. The organisation may release a claimed pickup, and an administrator's expiry sweep closes overdue, unaccepted donations. Every transition appends a server-stamped event.

1.1.5 Scope

Implemented: registration and sign-in, with the first administrator created from the command line; role, ownership, lifecycle and verification checks; donation posting with photo resizing; explainable ranking; the nine-state lifecycle with race-safe writes and an event ledger; standing requirements and a donor needs board; verification, the expiry sweep and metrics; four web portals; rate limiting; and automated tests in continuous integration.

Partially implemented: cancellation, pickup release, account management and setting an organisation's map coordinates work only through the API. Phone-layout screens at `/m/*` can be reached only by typing their address.

Not built: notifications, maps and routing; object storage for photos; scheduling of the expiry sweep; any deployment or real-user pilot; machine learning or computer vision; and certification of food safety. The platform records what a recipient needs in order to decide, but does not vouch for the food [GSHS26].

1.2 Project Objectives

Each objective states a measurable criterion and its verified mid-term status.

1. **Objective 1:** Donors post surplus food with an absolute, server-validated deadline. *Met.*
2. **Objective 2:** Eligible organisations are ranked by a published weighted heuristic that returns per-criterion scores and reasons. *Met.*
3. **Objective 3:** The lifecycle refuses illegal and conflicting transitions and records one event per transition. *Met; covered by lifecycle and concurrency tests.*
4. **Objective 4:** Every read and write is restricted by role, ownership, state and verification, and sensitive data is withheld from readers who do not need it. *Met; covered by scope tests.*
5. **Objective 5:** Organisations publish standing requirements that donors can browse. *Met.*
6. **Objective 6:** Time-to-claim, rescue rate, expiry-loss rate and handover time are derived from the event ledger. *Instrumented; not yet measured on real use.*
7. **Objective 7:** Automated tests run in CI on every push. *Met: 495 backend and 156 frontend tests pass.*
8. **Objective 8:** A staging deployment and pilot before week 17. *Not started.*

1.3 Organisation of the Report

The report follows the TIET report template [Tha26]. Chapter 2 covers the methodology, architecture, requirements, design and implementation modules. Chapter 3 presents testing, results and limitations, and Chapter 4 is the Feasibility Report. Chapter 5 concludes, and Chapter 6 contains the use case diagram and scenarios, the class diagram and the activity diagram.

Methodology and Design

FoodLink was built iteratively [GSHS26; Som16], as small numbered tasks (the latest is Task 46), each with its own regression tests, and every non-trivial choice was recorded in a decision log cited below by number (D-01 to D-61) [Foo26a]. Three audits of the running system (2, 5 and 10 September 2026) ranked findings from P0 to P3; all five P1 findings are fixed.

2.1 System Architecture

2.1.1 High-Level Design

FoodLink has three tiers: a React single-page application, a stateless REST API built with FastAPI, and a relational database (SQLite by default) reached only through the SQLAlchemy ORM. The API is the sole authority for every rule, makes no outbound requests (no email, SMS, maps or object storage) and runs no background workers. Five routers (`auth`, `donations`, `organisations`, `admin`, `metrics`) call shared modules with no HTTP logic (`matching`, `serialize`, `security`, `ratelimit`); there is deliberately no service layer (D-07).

2.1.2 Technical Stack

Layer	Technologies
Backend	Python 3.13 (3.10 or later required); FastAPI ≥ 0.115 [Ram26], uvicorn, Pydantic 2, PyJWT, bcrypt
Data	SQLAlchemy 2.0 [SQL26a]; Alembic ≥ 1.13 ; SQLite by default [SQL26b]
Frontend	React 18.3 [Met26], TypeScript 5.5, Vite 5.4, React Router 6, Tailwind CSS 3.4
Testing and CI	pytest with FastAPI's test client; Vitest 3.2 with Testing Library; GitHub Actions

Table 2.1: Technology stack, from the dependency manifests

2.1.3 System Requirements

FoodLink needs Python 3.10 or later, Node.js with npm (CI uses Node.js 20) and a web browser; no database server is needed, because migrations run at start-up. A signing key of at least 32 characters must be set in `FOODLINK_SECRET_KEY`, or the backend, CLI and Alembic refuse to start (D-22); no hardware specification has been defined or measured.

2.1.4 Functional Requirements

Table 2.2 lists the implemented functional requirements. *[API]* marks behaviour that is implemented and tested in the API but not yet offered in the portal.

ID	Requirement (status)
FR-01	Self-registration as donor, organisation or courier; sign-in; own profile and password
FR-02	A donor posts food details, an optional photo, a pickup pin and a future deadline
FR-03	Posting ranks eligible organisations and records MATCHED if any exist
FR-04	Verified organisations browse open donations with a live match explanation; readers can fetch the ranked list <i>[list: API, mobile]</i>
FR-05	A verified organisation accepts an open donation; overdue donations are refused
FR-06	A courier sets availability, claims a pickup (single winner) and records pickup and delivery
FR-07	The accepting organisation confirms receipt
FR-08	A donor cancels before pickup; the accepting organisation releases a claimed pickup <i>[API]</i>
FR-09	Organisations manage standing requirements; donors browse the needs of verified organisations
FR-10	Organisations edit their profile; a name or map-pin change voids verification <i>[pin: API]</i>
FR-11	Administrators verify or revoke organisations, run the expiry sweep, export donations as CSV, and manage accounts <i>[accounts: API]</i>
FR-12	Every transition is recorded with actor and server time; metrics derive from these records

Table 2.2: Functional requirements

2.1.5 Non-Functional Requirements

Security, **privacy** and **integrity** dominate: every request is authenticated and checked server-side for role, ownership, state and trust; reads are scoped in SQL and pickup locations and distances are withheld from readers who do not need them; and status changes are conditional writes recorded in an append-only, UTC-stamped ledger. The system must also be **explainable** (published weights, per-criterion reasons), **abuse-resistant** (rate limits, bounded inputs), **usable** (short forms, photo resizing, readable errors) and **maintainable** (typed schemas, rules expressed as data, a decision log and 651 automated tests).

2.1.6 Database Design

Six tables, created by one Alembic revision (`ae4636b1e6d4`), carry the workflow (Table 2.3). Attributes and multiplicities appear in the class diagram (Figure 6.2).

Table	Role in the workflow
<code>users</code>	Every account, with <code>role</code> as an enumeration; at most one recipient profile and one courier profile each
<code>recipients</code>	Organisation profile: optional coordinates, capacity in meals, <code>is_verified</code> , acceptance and completion counters
<code>volunteers</code>	Courier profile: base location, availability, completed deliveries
<code>donations</code>	Food details, exact pickup pin, absolute deadline, <code>status</code> , frozen <code>match_score</code> ; recipient and courier are null until accepted and claimed
<code>status_events</code>	Append-only history: previous and new state, actor, note and server time
<code>requirements</code>	An organisation's standing needs; <code>is_active</code> is their whole lifecycle

Table 2.3: Database tables and their role

Events replace timestamp columns because they record who acted and allow a state to be entered more than once (D-01). Reliability is computed, not stored: 85 until three acceptances, then completed divided by accepted (D-15); the frozen `match_score` is the deliberate stored exception (D-30). Counters change in the same transaction as their transition. Invariants live in application code, since SQLite does not enforce foreign keys by default.

2.1.7 API and Backend Design

The API exposes 27 JSON routes under `/api`, with camelCase fields and OpenAPI documentation at `/docs`; Table 2.4 summarises the resource groups.

Group	Representative routes	Callers
Auth	POST /auth/register, POST /auth/login, /auth/me, POST /auth/password	Anonymous (rate-limited); any signed-in user
Donations	POST /donations, GET /donations, /donations/{id}, /donations/{id}/matches and /status	Donor or admin to post; scoped readers; roles per target state
Organisations	/recipients, /recipients/me, /requirements, /requirements/{id}, /volunteers, /volunteers/me	Scoped by role and ownership
Metrics, Admin	GET /metrics; /admin/users, /admin/recipients/{id}/verify, /admin/maintenance/expire	Any signed-in user; administrators only

Table 2.4: API resource groups (paths relative to /api; plus /api/health)

Every error carries a human-readable sentence that the frontend shows verbatim (D-18). Status codes are consistent: 401 means an invalid token, 403 an insufficient role or trust, 404 not found *or not visible*, 409 a state conflict, 422 invalid input and 429 a rate limit, and a denied read never confirms that a record exists (D-24, D-26). In a PATCH an omitted field is untouched and `null` clears only nullable columns (D-61), and role and verification are absent from self-service schemas (D-16).

2.1.8 Frontend Architecture

`lib/api.ts` is the only network module; it attaches the token, surfaces the server’s error sentence and signs the user out on a 401 (D-12). `AuthContext` keeps the token in `localStorage`, and `AppContext` loads up to 500 donations plus role-specific data, reloading after every write. Four portals (`/donor`, `/ngo`, `/volunteer`, `/admin`) sit behind route guards that are a convenience only, because the API enforces access (D-14), and phone-layout screens under `/m/*` exist but nothing links to them (D-20).

2.2 Implementation Details

2.2.1 Module 1: Core Functionality — Donation Lifecycle

A donation passes through nine states: `AVAILABLE`, `MATCHED`, `ACCEPTED`, `VOLUNTEER_ASSIGNED`, `PICKED_UP`, `DELIVERED`, `COMPLETED`, `CANCELLED` and `EXPIRED`. The rules are data (D-02): `ALLOWED_TRANSITIONS` lists the legal moves, and anything else returns 409, while `TRANSITION_ROLES` lists who may cause each target state, and anyone

else receives 403. Pickup, delivery, completion, cancellation and release also re-read the donation through the caller's own read scope, so a non-party receives 404 (D-34, D-35); Table 2.5 summarises them.

Transition	Who	Key rules and side effects
(new) → AVAILABLE	Donor, admin	Future deadline; donor rate limits; bounded input
→ MATCHED	Automatic at posting	At least one eligible organisation; top score frozen
AVAILABLE/MATCHED → ACCEPTED	Verified organisation (for itself), admin	Refused if overdue (409); counts an acceptance; freezes its score
ACCEPTED → VOLUNTEER_ASSIGNED	Courier	Conditional claim; the loser receives 409
VOLUNTEER_ASSIGNED → ACCEPTED	Accepting organisation, admin	Release: courier cleared; not a new acceptance (D-41)
→ PICKED_UP → DELIVERED	Assigned courier, admin	Owned by the courier bound to the run
DELIVERED → COMPLETED	Accepting organisation, admin	Increments completion counters
→ CANCELLED	Owning donor before pickup; admin also after	Withdraws a counted acceptance (D-58)
→ EXPIRED	Admin	Status route, or the sweep over overdue AVAILABLE/MATCHED donations

Table 2.5: Lifecycle transitions and enforced rules

Donors may post 10 donations per hour per account and 30 per client address, with administrators exempt (D-59); free text is capped at 2,000 characters, and `imageUrl` must be an image data URL of at most 262,144 characters or an http(s) link (D-56, D-60).

A read-check-write sequence once let two organisations both accept the same donation; this lost update was reproduced on SQLite before being fixed [Kle17]. Every status change is now one statement, `UPDATE donations SET status = :to WHERE id = :id AND status = :from`. A row count other than one returns 409 and rolls back all side effects (D-55), and the courier claim binds the courier the same way (D-28). `SELECT ... FOR UPDATE` was rejected because SQLite ignores it.

The expiry sweep is triggered manually and closes overdue AVAILABLE and

MATCHED donations with actor-less events. Each **StatusEvent** records the previous and new state, actor, optional note and server time, and drives both user timelines and platform metrics.

2.2.2 Module 2: Core Functionality — Recipient Matching and Explainability

matching.py ranks organisations with a transparent weighted-sum heuristic; it is *not* a learned model (D-05). An organisation is excluded entirely, rather than scored low, if it is unverified, has no coordinates, or lies beyond the radius R (8 km by default) (D-06). Let d be the haversine distance in km [Sin84], q the quantity, c the capacity in meals, $r = q/c$, m the minutes to the deadline, and a and k the organisation’s accepted and completed counts. Each criterion in Table 2.6 scores 0–100, and

$$S = \text{round} (0.25 S_D + 0.25 S_Q + 0.20 S_C + 0.15 S_T + 0.15 S_R) . \quad (2.1)$$

Criterion	Weight	Score
Distance S_D	0.25	$100(1 - d/R)$; 0 at or beyond R
Quantity fit S_Q	0.25	Meals only: $40 + 60r$ if $r \leq 1$, else $\max(0, 100 - 120(r - 1))$
Capacity headroom S_C	0.20	Meals only: $100 \min(1, (c - q)/100)$; 0 if no room remains
Deadline comfort S_T	0.15	$100 \min(1, (m - t)/120)$ with travel time $t = 60d/20$ minutes
Reliability S_R	0.15	85 below three acceptances, else $100k/a$

Table 2.6: Matching criteria and published weights

For units other than meals, $S_Q = S_C = 50$ for every candidate, so size cannot re-order the ranking, and the explanation says size was not assessed (D-42). Candidates are ordered by S , then by how well one of their active standing requirements fits the donation (same unit only), then by urgency, and finally by id (D-52). Requirements break ties but never change a score, which is visible to readers who may not read requirements. Every result carries the five sub-scores and reasons from fixed thresholds, such as “2.0 km away in a straight line — well inside the collection radius”.

As a worked example, for 60 meals due in three hours the repository’s **score_pair** scores a new kitchen 2 km away with capacity 100 as $0.25(75) + 0.25(76) + 0.20(40) + 0.15(100) + 0.15(85) = 73.5$, rounded to 74; a kitchen 5 km away with capacity 250 and 9 of 10 completions scores 72, and unverified or 9 km-distant kitchens are excluded.

The frozen **matchScore** records the decision: it is written at posting and replaced at first acceptance, whereas **viewerMatch** is an organisation’s own live score for an

open donation (D-30). For readers other than that organisation or an administrator, positions are snapped to a 0.01° grid and `distanceKm` and the frozen score are withheld, which prevents triangulating private addresses (D-45, D-47); eligibility is still decided on the true position.

2.2.3 Module 3: Core Functionality — Authentication, Authorization and Trust

Self-registration refuses the administrator role at the schema level (D-04), and an organisation account starts *unverified*. Passwords of 8–128 characters are stored as bcrypt hashes [PM99], and sign-in returns an HS256 JSON Web Token valid for 720 minutes [JBS15]. Every request re-reads the user row and ignores the token’s role claim, so a suspension takes effect immediately (D-03). Lockout guards stop administrators from demoting themselves or the last active administrator.

Every operation passes four authorization layers: **role** (403); **ownership**, as a SQL clause (404 or an empty list); **lifecycle legality** (409); and **trust**, meaning administrator verification. FoodLink also controls *what* a record reveals to each reader (Table 2.7).

Data	Admin	Donor	Organisation	Courier
Donations readable	All	Own	Own, plus the open pool if verified	Unclaimed pickups and own runs
Pickup pin, address, donor	All	Own	All it can read	Own claimed runs only
Frozen score, true distance	All	None	Own organisation	None
Contact directories (phones)	All	None	Own row; own couriers	None
Standing requirements	All	Verified organisations’ needs	Own	None

Table 2.7: Reader-specific read and disclosure scopes

Verification is the trust boundary: unverified organisations are not ranked, cannot see the open pool and cannot accept (D-37, D-53). It is read on every request, so revoking it takes effect at once, and a real change to an organisation’s name, latitude or longitude clears it in the same commit (D-54). Couriers see only a coarse `pickupArea` until a claim binds the pickup to them (D-57).

2.2.4 Module 4: Secondary Features — Standing Requirements

An organisation posts needs with a food type, quantity and unit, beneficiaries, urgency, a recurrence flag and notes. One flag, `is_active`, is their whole lifecycle (D-29): false retires a need but keeps the row, true reopens it. Ownership is enforced in the query, so another organisation's need returns 404, and while donors see active needs of verified organisations on the *Needs Board*, couriers see none (D-44, D-46). In ranking, requirements only break ties and add a reason.

2.2.5 Module 5: Secondary Features — Administration and Platform Metrics

All administration routes sit behind one role dependency. Administrators verify or revoke organisations, filter and export donations as CSV, run the expiry sweep, and view the courier roster and analytics; account management is API-only. They also appear in every entry of `TRANSITION_ROLES`, so they can act on a party's behalf under the same rules. `GET /api/metrics` derives volume totals and, from the event ledger, median **time-to-claim** (creation to first `ACCEPTED`), median **handover time** (`ACCEPTED` to `DELIVERED`), the **rescue rate** (completions by the deadline over completed plus expired donations) and the **expiry-loss rate** (expired over that same total). Role impact pages use the account's own donations (D-32).

2.2.6 Module 6: Secondary Features — Role Portals and User Workflows

Donors post on *Create Food Donation* (geolocation or typed coordinates, as there is no map picker), follow *My Donations* and browse the *Needs Board*. Verified organisations use *Available Donations*, *Accepted Deliveries* and *Food Requirements & Demand Signals*; couriers use *Pickup Tasks*, and administrators the *Partner Organizations Directory* and *Platform Donation Ledger*.

2.2.7 Security and Privacy Considerations

The controls above address this domain's common web-application risks [OWA21] — broken access control, privilege escalation, credential attacks, location exposure, tampered history and abusive posting — but residual risks remain: no token revocation, refresh token, MFA or content security policy; registration reveals whether an email exists and transition errors reveal a donation's status; photos, descriptions and notes are bounded but not scoped; and the radius gate is a rate-limited membership oracle. Section `efsec:limitations` lists the remaining gaps.

Results and Evaluation

FoodLink has not been deployed or used by real donors, organisations or couriers. Its mid-term evaluation therefore rests on *verification*: checking that the behaviour matches the requirements, and that the security, privacy and integrity properties hold under adversarial and concurrent use.

3.1 Testing Methodology

Every audit fix arrived with regression tests that fail without the fix [PM15]. **Unit tests** exercise the backend matcher’s formulas, gates, unit handling, blurring and tie-break directly, together with configuration and the rate limiter. On the frontend, Vitest covers the library modules for adapters, time, location display, impact, the HTTP client and photo resizing.

Integration tests make up most of the backend suite. They drive the full HTTP stack against a fresh in-memory SQLite database with no mocks (D-17), and try every role against every scoped read and transition. Concurrency tests interleave two real transactions on a file database to reproduce, and then refuse, lost updates. Migration tests compare a database built from the Alembic history with the models, and frontend component tests check both what screens show and what they must not show. **Performance testing** has not been performed, so no load, stress or response-time data exist. **User testing** has not been conducted either; the proposal’s pilot [GSHS26] remains future work.

The 27 backend test files are grouped by the property they protect: accounts and configuration, input validation, read and disclosure scopes, lifecycle and concurrency, and matching and location privacy. On every push, a GitHub Actions workflow runs two jobs. The backend job runs `pytest`, then `alembic upgrade head` and `alembic check`, to catch models committed without a migration. The frontend job runs `npm test` and `npm run build`, which includes the TypeScript compiler. CI deploys nothing and holds no secrets (D-25).

3.2 Performance Metrics

The checks in Table 3.1 were run on the repository at commit `e588b35` on 15 September 2026, with the backend database redirected to a temporary file.

The evidence has gaps. The production build was not re-run for this report, because it writes into the repository; CI runs it on every push. No response-time or throughput figures exist. The proposal’s outcome metrics [GSHS26] are computed by

Metric	Target	Achieved
Backend tests (<code>pytest</code>)	All pass	495 passed, 0 failed (337 s)
Frontend tests (<code>npm test</code>)	All pass	156 passed in 18 files
Type check (<code>tsc --noEmit</code>)	No errors	No errors
Migration drift (<code>alembic check</code>)	No drift	At head <code>ae4636b1e6d4</code> ; no changes detected
P1 audit findings (10 Sep 2026)	All fixed	5 of 5, each with regression tests

Table 3.1: Validation results at mid-term

GET `/api/metrics` but have no real data yet: median time-to-claim (target ≤ 45 minutes), rescue rate (target $\geq 80\%$), expiry-loss rate (which also depends on the manual sweep having run) and handover time. User-clarity ratings are not instrumented, and service availability cannot be measured while nothing is deployed.

3.3 Analysis

Objectives. Objectives 1–5 and 7 are met. Objective 6 exists only as instrumentation, and Objective 8 (deployment and pilot) has not started.

Beyond the proposal. Ranking and requirements, planned as later deliverables, are both implemented. So are properties the proposal did not name: reader-specific disclosure, race-safe transitions, abuse limits, and reliability accounting that cancellations cannot distort.

Challenges. The hardest problems appeared only under adversarial or concurrent use. Each was reproduced against the running system, fixed with the smallest change that made the property hold everywhere, and recorded as a decision (Table 3.2). None needed a schema change.

3.3.1 Current Limitations

1. **Not deployed, no real-user pilot:** there is no deployment configuration, PostgreSQL is untested, and no outcome or performance data exist.
2. **Missing portal controls:** cancellation, release and account management work only through the API. No screen sets an organisation’s map coordinates, so an organisation created through the portal or by an administrator cannot be ranked until its pin is set through `PATCH /api/recipients/me`. In practice, only organisations with coordinates (such as the seeded ones) appear in rankings.

Challenge (audit finding)	Resolution
New unverified accounts could read open donations' exact pins and donor names (P1-1)	Open pool requires verification (D-53); couriers see a coarse area until they claim (D-57)
Verification survived an organisation renaming or moving itself (P1-2)	A real name or pin change voids verification (D-54)
Two organisations accepting simultaneously both succeeded (P1-3)	Conditional status writes; the loser receives 409 and its changes roll back (D-55)
Donor cancellations lowered an organisation's reliability (P1-4)	Cancellations withdraw the counted acceptance; no donor cancellation after pickup (D-58)
Ordinary phone photos exceeded the image cap (P1-5)	Browser-side resizing before upload (D-56)
Exact distances allowed triangulating organisations (HA-3)	Blurred positions; distances and scores withheld (D-45, D-47)

Table 3.2: Principal challenges found by audit and their resolution

3. **No notifications or maps:** users must open the portal to see changes. Distances are straight-line with a flat 20 km/h travel estimate, and pins come from geolocation or typed coordinates.
4. **Manual expiry:** the sweep's write is unguarded, and an administrator's ACCEPTED → EXPIRED leaves the acceptance counted.
5. **Sessions:** no revocation, refresh token, MFA or CSP, and rate-limit counters are per process.
6. **Heuristic weights:** the matching weights and reliability prior are judgement, not tuned on data.
7. **Scale:** photos are stored inline, every write reloads up to 500 donations, ranking and metrics run in Python, and SQLite allows one writer.
8. **Validation gaps:** profile text is unbounded, and unexpected errors return an empty 500 response.
9. **Mobile residue:** the phone camera screen simulates a photo read with a fixed address, one mobile profile shows a hard-coded email, and the `/ngo/available/:id` deep link does not select a donation.
10. **Metric caveat:** the completed-meals total adds quantities across units.

Feasibility Report

A feasibility study asks whether a system can be built and used under real technical, operational, economic, schedule and legal constraints [Som16; PM15]. Much of FoodLink already exists, so this chapter draws on evidence from the implementation. For each dimension it separates what is **implemented**, what is **feasible but not implemented**, and what remains **future work**.

4.1 Technical Feasibility

The stack (Table 2.1) consists of mature, well-documented open-source components, and the project needed no experimental technology and no external service. The strongest evidence of feasibility is that the complete workflow runs today and passes 651 automated tests.

On the **frontend and backend**, four React portals and 27 stateless REST routes, whose rules are inspectable data, are implemented. Feasible but not yet built are portal controls for the API-only operations, routing phone visitors to `/m/*`, a global exception handler, profile bounds, and a scheduled caller for the expiry sweep, which is already an idempotent endpoint. The **database** consists of six tables under Alembic migrations on SQLite, which needs no database server. PostgreSQL is feasible because all access goes through SQLAlchemy and the concurrency guards are plain conditional updates. However, no driver is pinned, the tests have never run on PostgreSQL, and the guards rely on its default READ COMMITTED isolation (D-28). SQLite permits one writer [SQL26b], which suits a single-process pilot but not several workers. For **authentication**, bcrypt, expiring signed tokens and a fail-closed key are implemented. Token revocation and security headers are feasible next steps, while email verification and MFA are future work.

Deployment. No deployment configuration exists, but a single-process SQLite deployment is feasible now under five constraints. Migrations run at start-up, which is safe with one process only (D-23). Rate-limit counters live in process memory (D-27). Behind a proxy, the forwarded client address must be trusted explicitly. The deployed origin must be added to the CORS allowlist and the signing key supplied as a secret. Finally, the expiry sweep needs an external scheduler.

Complexity. The domain is small (six tables, nine states), but the implementation is not trivial: SQL read scopes, reader-specific disclosure, race-safe transitions and location privacy. The team delivered it through small, tested increments, and the remaining work is algorithmically simpler.

Assessment: technically feasible, and already demonstrated for the core system.

The main open risk is the untested production database and deployment topology.

4.2 Operational Feasibility

Donors complete a single form with automatic photo resizing, but have no map picker, push updates or cancel button. **Organisations** first wait for verification, a deliberate delay that requires genuine administrator vetting. After that they accept with one action, confirm receipt on *Accepted Deliveries* and maintain needs on *Food Requirements & Demand Signals*. The serious gap is that no screen sets an organisation's map coordinates, so a portal-registered organisation is never *suggested* to donors until its pin is set through the API. **Couriers** judge runs from a coarse area, claim one, and record pickup and delivery with one button each. Availability is advisory, and only the organisation (via the API) or an administrator can release a claimed run. **Administrators** verify organisations, monitor donations, run the sweep and review analytics in the portal. Managing accounts through the API or command line is workable only for a small, technically supervised pilot.

Every point of custody keeps a human decision (accept, claim, confirm), and nothing is assigned automatically [GSHS26]. The main risk is timeliness without notifications.

Assessment: operationally feasible for a supervised campus pilot. Unsupervised use needs notifications, a map-pin field and the missing controls.

4.3 Economic Feasibility

No hosting provider or pilot scale has been chosen, so no monetary figures are given. Development cost is the four-member team's time within the course. All software dependencies are open source with no licence fees, and the only services used are a Git hosting platform with CI and an online LaTeX editor. There is no infrastructure cost today, since nothing is deployed and the backend calls no paid APIs. A pilot would need a small application server and, later, a managed database. Notifications, photo storage and routing would add usage-based charges, which have not been estimated. SQLite, straight-line distance and browser-side photo resizing were chosen partly because they keep costs down. On the benefit side, rescue rate and expiry-loss rate become measurable once the system is used; no benefit figures are claimed.

Assessment: economically feasible at course and pilot scale.

4.4 Schedule Feasibility

The course requires a proposal in week 4, this report in week 7 and a final report in week 17 [Ven26]. The first commit is dated 29 July 2026, and 85 commits existed by 13 September 2026 (e588b35). The lifecycle, matching, requirements and authentication are complete for a pilot, and the web portals cover the core flows. Phone layouts and frontend test coverage are partial, and deployment and the pilot have not started. The remaining course work, in priority order, is: (1) the P2 items (exception handler, profile bounds, deep link, frontend tests and mobile residue); (2) portal controls for the API-only operations, plus a map-pin field; (3) a deployment with a scheduled sweep; (4) a short pilot. Notifications, road distances, photo storage and weight tuning lie beyond the course.

Assessment: schedule-feasible. The core was delivered by mid-term, and the main risk is recruiting real participants for a pilot.

4.5 Legal, Security and Privacy Feasibility

FoodLink stores names, emails, phone numbers, organisation addresses, donors' pickup locations, photos and activity history. Any deployment in India would have to comply with the Digital Personal Data Protection Act, 2023 [Min23]; this report gives no legal opinion.

Authentication, four-layer authorization, verification and location-disclosure controls are implemented (Section 2.2.3). Session revocation and MFA are missing. Descriptions, photos and notes are not scoped, so an address typed into a description would be exposed. Before public use, FoodLink needs a privacy notice and consent at sign-up; ways to obtain, correct or delete personal data (accounts can currently only be deactivated); a retention policy; and protected hosting and backups. On food safety, the platform records storage type, preparation time and deadline but does not certify food, and inspection rests with the recipient [GSHS26]. Terms of use stating this, and a review of food-safety obligations, are still needed.

Assessment: suitable for a supervised pilot with informed participants. Public operation requires the privacy, consent, data-rights and terms measures above.

4.6 Feasibility Summary

Table 4.1 summarises the five dimensions.

Dimension	Implemented	Feasible, not built	Future work	Verdict
Technical	Full workflow; 27 routes; 651 passing tests	PostgreSQL, scheduler, revocation	Multi-process deployment	Feasible
Operational	Four portals covering every core step	Pin field; cancel, release and account screens	Notifications	Feasible for pilot
Economic	Open-source stack; no paid services	Single-server pilot hosting	Messaging and storage costs	Feasible
Schedule	Core system by mid-term	P2 fixes; deployment configuration	Pilot and metrics	Feasible
Legal and privacy	Scoped access; location protection; verification	Security headers; profile bounds	Consent, data rights, retention, terms	Conditional

Table 4.1: Summary of feasibility by dimension

Conclusion

5.1 Summary of Work

FoodLink set out to shorten the time between surplus food being posted and a trustworthy organisation accepting it, and to see the food through delivery with a record of every step. By mid-term, the team has built and verified the core system, meeting Objectives 1–5 and 7. The system comprises a FastAPI backend (27 routes, six tables) and a React frontend with four role portals. Donations follow a nine-state lifecycle with conditional writes and an append-only event ledger. An explainable weighted matcher ranks organisations, using requirements only to break ties. Role, ownership, lifecycle and verification controls are enforced, with disclosure tailored to each reader. Standing requirements, ledger-derived metrics and administrator tools complete the feature set, and 495 backend and 156 frontend tests pass in CI. Objective 6 exists as instrumentation awaiting real use, and deployment and a pilot (Objective 8) have not started. The limitations in Section 3.3.1 remain open, notably the missing portal controls and the organisation map-pin gap.

5.2 Key Findings

Finding 1: *a transparent heuristic is sufficient and verifiable.* Rankings can be recomputed by hand, and tying explanations to the scoring code exposed a real defect: two criteria that were effectively one. **Finding 2:** *most of the difficulty lies in access and integrity, not features.* The most serious defects were unverified accounts reading donors' addresses, verification that survived a change of identity, and double acceptance, all invisible in single-user testing. **Finding 3:** *race conditions are real even on SQLite.* Moving each precondition into the write itself fixed them testably. **Finding 4:** *privacy needs reader-specific answers.* Distances and scores can reveal the locations they come from, so what a record says must depend on who reads it.

5.3 Future Work and Recommendations

None of the following is implemented. Recommended order:

1. close the remaining audit items: an exception handler, profile bounds, a guarded expiry sweep, the deep-link fix, removal of the mobile residue, and frontend tests;

2. complete the portal with a map-pin field for organisations and controls for cancel, release and account management;
3. deploy with a secret key, a scheduled sweep and, if needed, PostgreSQL; then run a campus pilot to measure time-to-claim, rescue rate and handover time;
4. add notifications, token revocation, security headers and an administrator audit log;
5. add a map-based pin picker, object storage for photos, pagination and database-side aggregation;
6. extend matching with structured food categories and road distance. Weight tuning should wait for outcome data, and any learned ranker would sit behind `score_pair`, must remain explainable, and must show measured improvement;
7. prepare legally: a privacy notice and consent, data-access and deletion requests, retention rules, and terms of use.

5.4 Final Remarks

FoodLink has moved from a proposal to a working, tested system whose rules can be read, whose rankings can be checked by hand, and whose history cannot be rewritten by its users. The most valuable lessons came from treating the system adversarially. What remains before the final evaluation is to put it in front of real users and let the event ledger measure whether surplus food reaches people before it expires.

System Design: UML Diagrams

This chapter presents FoodLink’s design in UML [Obj17]: the use case diagram with scenarios, the class diagram and the activity diagram. Every element was derived from the repository at commit `e588b35`. Actors are the values of `UserRole`, use cases are implemented routes and screens, classes are the ORM models and the matching module, and activity branches are decisions the server takes. The earlier copies in `docs/uml/` predate decisions D-50 to D-58; the diagrams here are corrected redraws, with TikZ sources in the report’s `diagrams/` directory.

6.1 Use Case Model

6.1.1 Use Case Diagram

Figure 6.1 shows four actors and 26 use cases. The actors *Donor*, *Recipient Organisation (NGO)*, *Volunteer Courier* and *Admin* specialise an abstract *Platform User*, which carries sign-in, account management and status tracking. *Register Account* is open only to the donor, organisation and courier roles. *Post Donation*, *View Ranked Matches*, *Browse Open Donations* and *Accept Donation* each `«include»` the internal *Rank Recipient Organisations*, and *Review Match Explanation* `«extend»`s *Browse Open Donations*. Some conditions are not drawn: browsing and accepting require a verified organisation, overdue donations cannot be accepted, and donors cancel only before pickup. *Override Lifecycle Transition* represents the administrator’s place in every entry of `TRANSITION_ROLES` except the courier claim.

6.1.2 Use Case Scenarios

The fifteen main use cases are specified below. Status codes are the server’s responses, and routes are relative to `/api`.

UC-01 — Register Account

Actor, goal Prospective donor, organisation or courier. *Goal:* an account in a self-service role.

Pre., trigger Email unused; under 10 registrations per hour from the network. *Trigger:* creating an account on the sign-in page.

Main flow (1) Picks a role; enters name, email and password. (2) `POST /auth/register` validates the email, an 8–128-character password and a self-signup role. (3) The server stores a bcrypt hash and creates a courier or *unverified* organisation profile. (4) It returns 201 with a token.

Alternatives Email exists: 409. **admin** role or invalid fields: 422. Over the limit: 429.
Post., rules Active account. An organisation is not ranked, cannot see the pool and cannot accept until verified (UC-13) and given coordinates (API only). Administrators never self-register (D-04).

UC-02 — Sign In

Actor, goal Any registered user. *Goal:* an authenticated session in the right portal.
Pre., trigger Account exists; under 30 attempts per 5 minutes from the network. *Trigger:* submitting email and password.
Main flow (1) `POST /auth/login` (OAuth 2.0 password form). (2) The server checks the bcrypt hash and that the account is active. (3) It returns a 720-minute HS256 token, and the client opens the role's portal. (4) Each later request re-reads the user.
Alternatives Wrong email or password: 401 (one message). Suspended: 403. Over the limit: 429. Expired token or suspension mid-session: 401 and client sign-out.
Post., rules Signed in until expiry; sign-out is client-side. The token's role claim is never trusted (D-03).

UC-03 — Post Donation

Actor, goal Donor (or administrator). *Goal:* offer surplus food to nearby verified organisations before it expires.
Pre., trigger Signed in; under 10 posts per hour for the account and 30 for the network. *Trigger:* submitting *Create Food Donation*.
Main flow (1) Enters food details, address, pin, optional photo (resized in the browser) and deadline. (2) `POST /donations` checks role, limits, field bounds and a future deadline. (3) The donation is stored as **AVAILABLE** with an event, and organisations are ranked. (4) If any is eligible, the top score is frozen and **MATCHED** recorded; 201 is returned.
Alternatives None eligible: stays **AVAILABLE**. Past deadline or invalid input: 422. Over budget: 429. Organisation or courier: 403.
Post., rules Open to verified organisations until the deadline; nothing is assigned. Frozen score and distances are hidden from donors (D-47).

UC-04 — View Ranked Matches and Match Explanation

Actor, goal Recipient organisation; donor and administrator read the ranked list. *Goal:* see how well a donation suits an organisation, and why.
Pre., trigger Organisation verified; donation open and not overdue. *Trigger:* selecting a donation on *Available Donations*, or `GET /donations/{id}/matches`.
Main flow (1) `GET /donations` includes `viewerMatch`, the organisation's live `score_pair` result from its true position and own active needs. (2) The panel shows the score, five sub-scores, distance and reasons. (3) Variant: `/matches` lists up to 25 eligible organisations in order.

- Alternatives** Unverified: pool not returned, with an explanation. Ineligible: no breakdown. Other readers: blurred positions, null `distanceKm`. Unreadable donation: 404.
- Post., rules** Read-only. Requirements affect only order and reasons (D-52); frozen and live scores differ (D-30).
-

UC-05 — Accept Donation

- Actor, goal** Recipient organisation (or an administrator for it). *Goal*: take responsibility for an offered donation.
- Pre., trigger** Organisation verified; donation AVAILABLE or MATCHED and not overdue. *Trigger*: *Accept Donation*.
- Main flow** (1) POST `/donations/{id}/status` with ACCEPTED. (2) The server checks transition, role, deadline, own organisation and verification. (3) It binds the organisation, counts one acceptance and freezes its score. (4) The conditional write and event commit; the donation joins *Accepted Deliveries* and the courier queue.
- Alternatives** Overdue: 409. Unverified: 403. Another organisation accepted first: 409, changes rolled back. Another organisation's id: 403. No profile: 422.
- Post., rules** Exactly one bound organisation and one counted acceptance (D-06, D-50, D-55).
-

UC-06 — Claim Pickup

- Actor, goal** Volunteer courier. *Goal*: take on collection and delivery.
- Pre., trigger** Courier profile exists; donation ACCEPTED and unclaimed. *Trigger*: *Accept Pickup* on *Pickup Tasks*.
- Main flow** (1) The queue shows food, deadline, organisation and only a coarse `pickupArea`. (2) The courier requests VOLUNTEER_ASSIGNED. (3) A conditional UPDATE binds the courier only if still ACCEPTED and unclaimed. (4) The event commits; exact pin, address and donor become visible.
- Alternatives** Another courier won: 409, "Another courier has already claimed this pickup". No longer ACCEPTED: 409. No courier profile: 422.
- Post., rules** Exactly one courier (D-28, D-57). Availability does not gate claiming; couriers are not verified.
-

UC-07 — Record Pickup and Delivery

- Actor, goal** The assigned courier. *Goal*: record collection, then delivery.
- Pre., trigger** Donation VOLUNTEER_ASSIGNED to, or PICKED_UP by, the caller. *Trigger*: *Mark Picked Up*, then *Mark Delivered*.
- Main flow** (1) Sends PICKED_UP. (2) The server checks transition and role and re-reads the donation through the courier's scope. (3) The conditional write and event commit. (4) DELIVERED follows the same way after handover.

Alternatives Another courier's run: 404. Out of order: 409. After pickup the donor cannot cancel nor the organisation release (409).

Post., rules Awaits confirmation; in the courier's history from DELIVERED (D-51).

UC-08 — Confirm Receipt

Actor, goal The accepting organisation. *Goal*: close the donation as received.

Pre., trigger Donation DELIVERED and bound to the caller. *Trigger*: *Confirm receipt* on *Accepted Deliveries*.

Main flow (1) Sends COMPLETED. (2) The server checks transition, role and ownership. (3) It increments the organisation's completions and the courier's deliveries. (4) The conditional write and event commit.

Alternatives Another organisation's donation: 404. Not yet delivered: 409. Donor or courier: 403.

Post., rules Terminal. Reliability updates for future rankings; counts towards the rescue rate if completed by the deadline.

UC-09 — Cancel Donation (API only)

Actor, goal Donor; secondary: administrator. *Goal*: withdraw a donation that will not be handed over.

Pre., trigger Donor owns it and it is not yet PICKED_UP (an administrator may also cancel from PICKED_UP). *Trigger*: status CANCELLED via the API; no portal button.

Main flow (1) The server checks transition, role, ownership and the donor's window. (2) The conditional write and CANCELLED event are applied. (3) In the same transaction, a bound organisation's counted acceptance is withdrawn (never below zero).

Alternatives Donor after pickup, or DELIVERED/COMPLETED: 409. Another donor: 404. A courier's pickup commits first: 409, nothing withdrawn.

Post., rules Terminal. Reliability is unaffected (D-58); no reason is recorded.

UC-10 — Release Pickup (API only)

Actor, goal The accepting organisation; secondary: administrator. *Goal*: return a claimed pickup to the courier queue.

Pre., trigger Donation VOLUNTEER_ASSIGNED and bound to the caller's still-verified organisation. *Trigger*: status ACCEPTED via the API; no portal button.

Main flow (1) The server checks transition, role and ownership, treating this acceptance as a release. (2) It confirms verification and the binding. (3) It clears the courier without a new acceptance or re-frozen score. (4) The conditional write and event commit.

Alternatives Not the bound organisation: 404. Already picked up: 409. Verification revoked: 403.

Post., rules Unclaimed again, shown to couriers as a coarse area (D-41). Known issue: an administrator release naming another organisation counts a second ac-

ceptance.

UC-11 — Manage Standing Requirements and Browse Needs Board

Actor, goal Recipient organisation; secondary: donor. *Goal:* publish needs before supply exists.

Pre., trigger Organisation profile exists (posting needs no verification). *Trigger:* *Food Requirements & Demand Signals*; a donor opens *Needs Board*.

Main flow (1) POST `/requirements` with food type, quantity, unit, beneficiaries, urgency, recurrence and notes. (2) PATCH `/requirements/{id}` edits or retires a need (`isActive: false`). (3) Retired needs are listed with `includeInactive` and can be reopened. (4) Donors see active needs of verified organisations.

Alternatives Another organisation's id: 404. Invalid values: 422, nothing written. Explicit `null`: unchanged. Courier: empty list.

Post., rules Rows are never deleted. Active needs may break ties and add a reason, never change scores (D-29, D-44, D-52).

UC-12 — Edit Organisation Profile (verification-sensitive)

Actor, goal Recipient organisation. *Goal:* keep the organisation's details correct.

Pre., trigger Organisation profile exists. *Trigger:* saving *NGO / Recipient Profile*.

Main flow (1) PATCH `/recipients/me` with name, type, address, capacity, contact or phone. (2) The server keeps real changes only; `null` clears only nullable fields. (3) If a verified organisation's name, latitude or longitude changes, `is_verified` is cleared in the same commit. (4) The page shows the result.

Alternatives Non-positive capacity or out-of-range coordinates: 422. Unchanged name or other fields only: verification kept. Coordinates are API-only; clearing the pin voids verification.

Post., rules A voided organisation is excluded from ranking, pool and acceptance until re-verified (D-54); no self-verification (D-16).

UC-13 — Verify or Revoke Organisation

Actor, goal Administrator. *Goal:* vouch for a genuine organisation, or withdraw that vouching.

Pre., trigger Signed in as administrator; organisation exists. *Trigger:* the verify control on *Partner Organizations Directory*.

Main flow (1) The page lists organisations with verification state, reliability and the number awaiting verification. (2) After checking outside the system, the administrator sends POST `/admin/recipients/{id}/verify`. (3) The server sets `is_verified`; the list refreshes. (4) DELETE on the same route revokes.

Alternatives Non-administrator: 403. Unknown organisation: 404.

Post., rules Verified with coordinates: ranked, sees the pool, can accept; revoked: loses these, accepted donations unaffected. Verification is a human judgement

(D-37); administrator-created organisations start verified.

UC-14 — Run Expiry Sweep

Actor, goal Administrator. *Goal*: record unaccepted, overdue donations as losses.

Pre., trigger Signed in as administrator. *Trigger*: running the sweep from *Platform Donation Ledger*.

Main flow (1) POST `/admin/maintenance/expire`. (2) The server selects AVAILABLE and MATCHED donations past their deadline. (3) It appends actor-less EXPIRED events (“Deadline passed with no recipient”) and sets the status. (4) It returns the count.

Alternatives Nothing overdue: count 0. Non-administrator: 403.

Post., rules Terminal; counted in the expiry-loss rate. Known issues: not scheduled, overdue ACCEPTED donations untouched, write not conditional.

UC-15 — Manage User Accounts (API only)

Actor, goal Administrator. *Goal*: create accounts of any role; suspend, restore, rename or re-role them.

Pre., trigger Signed in as administrator. *Trigger*: calls to `/admin/users`; no screen.

Main flow (1) GET lists accounts, optionally by role. (2) POST creates any role, including administrator, with its profile (organisations start verified). (3) PATCH `/admin/users/{id}` updates `isActive`, `role`, `name`, `organization` or `phone`.

Alternatives Duplicate email: 409. Suspending or demoting oneself or the last active administrator: 409. Unknown user: 404.

Post., rules Effective on the next request (a suspended user receives 401, and 403 at sign-in). The first administrator comes only from `python -m foodlink.cli create-admin` (D-04).

The remaining use cases in Figure 6.1 are simple. *Manage Own Account* uses PATCH `/auth/me` and POST `/auth/password`; *Track Donation Status* lists donations in one’s scope with their timelines; *Browse Unclaimed Pickups* is the coarse-area queue of UC-06; *Set Courier Availability* uses PATCH `/volunteers/me` and is advisory; *View Platform Metrics* calls GET `/metrics`; and *Override Lifecycle Transition* performs any transition in Table 2.5 except the claim, on a party’s behalf.

6.2 Class Diagram

Figure 6.2 shows the six persistent entities and two enumerations of `models.py`, and the `matching` module with its two data classes. Multiplicities follow the foreign keys: a user acts for at most one **Recipient** (seeded recipients have none) and at most one **Volunteer**, whose user key is unique and mandatory; a **Donation** has zero or one recipient and courier until acceptance and claim; and a **StatusEvent** optionally names

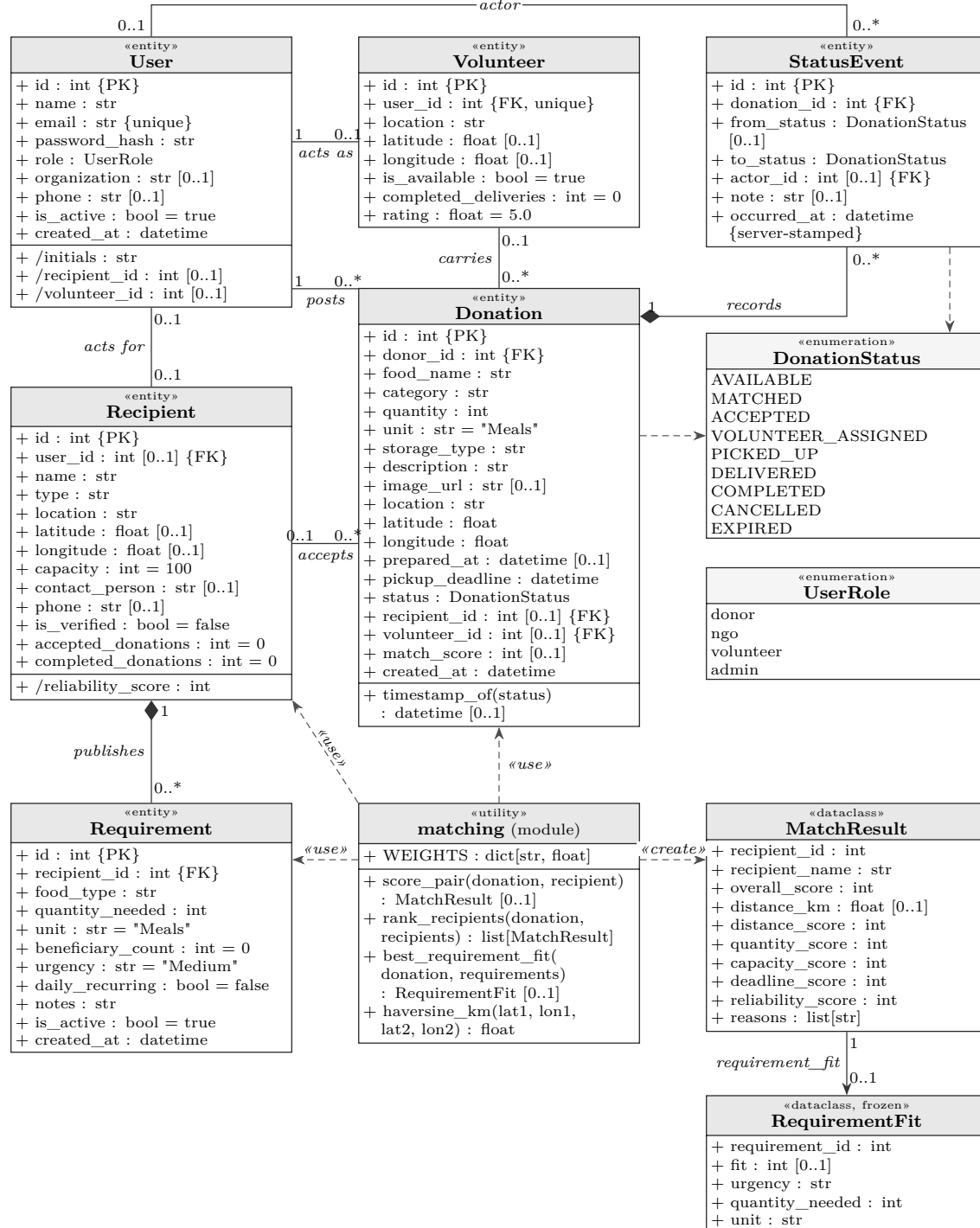


Figure 6.2: Class diagram of the FoodLink domain model and matching module

its actor. `Donation-StatusEvent` and `Recipient-Requirement` are compositions with cascading deletes, and derived properties carry a slash (for example `Recipient.reliability_score`). The matching module is a `«utility»` whose `MatchResult` objects are transient; only `overall_score` is copied into `Donation.match_score`. Roles are an enumerated attribute and profiles are associations, so there is no inheritance.

6.3 Activity Diagram

Figure 6.3 models the workflow across five swimlanes: the four roles and the server, the only lane that records state. The main path runs through validation, storage and ranking (`MATCHED` only when an organisation is eligible), acceptance gated by verification, the deadline and a conditional write, then a conditional courier claim, pickup, delivery and confirmation. Refused requests end at flow-final nodes and leave the donation unchanged. Donations nobody accepts in time end in `EXPIRED` through the sweep, and an API-only release loop returns a claimed pickup to the queue. The dashed interruptible region spans the states in which a donor may still cancel; its interrupting edge ends in `CANCELLED` and withdraws any counted acceptance. Administrator cancellation after pickup, administrator `ACCEPTED` → `EXPIRED`, and administrators acting for other parties are omitted for readability.

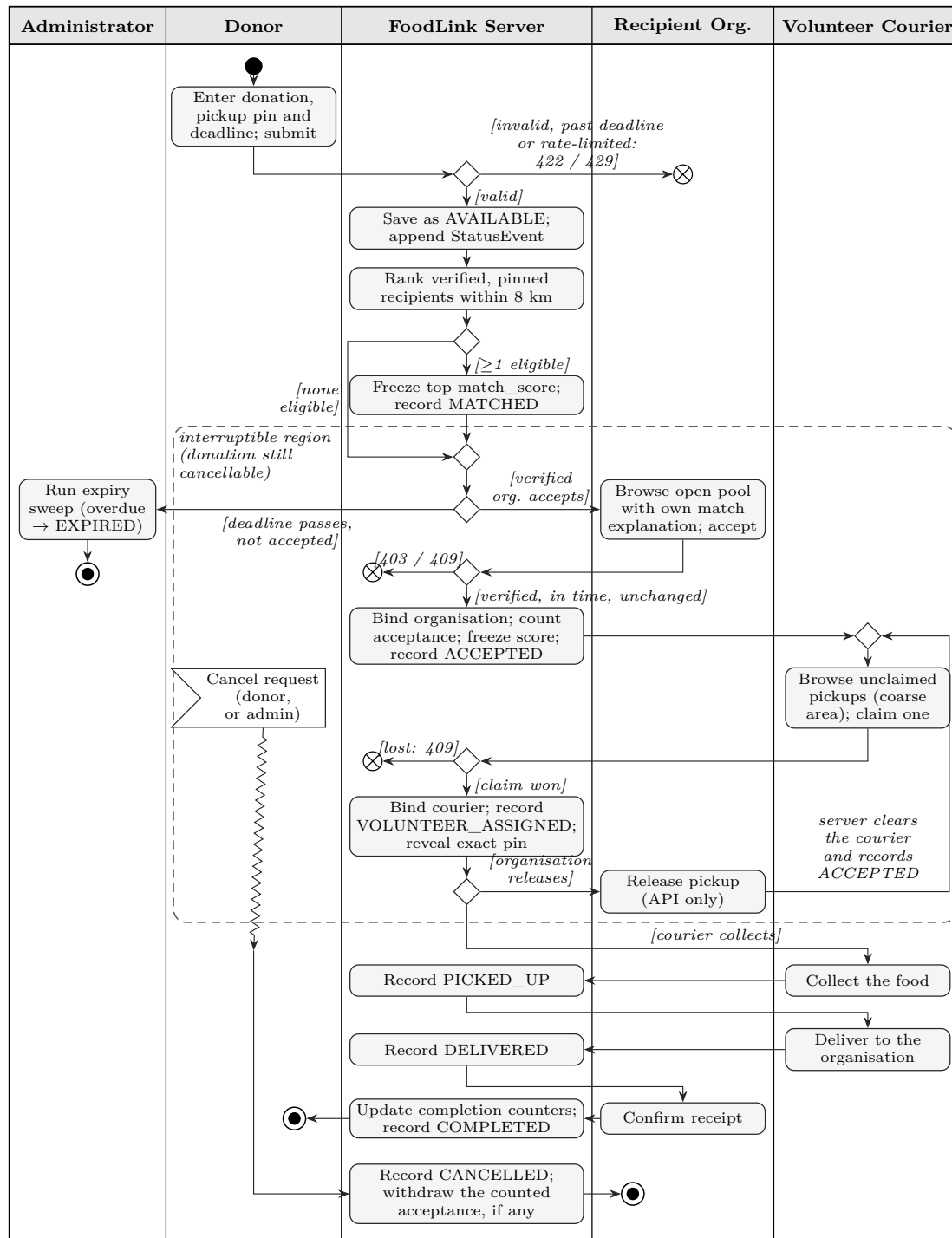


Figure 6.3: Activity diagram of the donation lifecycle, from posting to completion, expiry or cancellation. Codes on refusal edges: 422 invalid input or past deadline; 429 rate limit; 403 unverified organisation; 409 overdue, already taken or claim lost.

Bibliography

- [Foo26a] FoodLink Project Team. *FoodLink Architecture, Decision and Task Records*. `ai/ARCHITECTURE.md`, `ai/DECISIONS.md` (D-01 to D-61), `ai/TASKS.md` and `ai/PROJECT_STATE.md` in the FoodLink repository. 2026 (cit. on pp. 5, 9).
- [Foo26b] FoodLink Project Team. *FoodLink Source Repository*. Git repository, branch `master` at commit `e588b35`; backend in `code/foodlink/`, frontend in `frontend/src/`, tests in `code/tests/`. 2026 (cit. on p. 5).
- [GSHS26] Rishi Garg, Aayushi Sud, Yugam Heer, and Himanshi Singla. *FoodLink: A Coordination Platform for Surplus Food Redistribution — Project Proposal for UCS503P*. Project proposal, `project-proposal/main.tex` in the FoodLink repository. 2026 (cit. on pp. 5–7, 9, 17, 22, 23).
- [JBS15] Michael B. Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. Request for Comments 7519. Internet Engineering Task Force, 2015 (cit. on p. 15).
- [Kle17] Martin Kleppmann. *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017 (cit. on p. 13).
- [Met26] Meta Open Source. *React Documentation*. 2026. URL: <https://react.dev/> (cit. on p. 9).
- [Min23] Ministry of Law and Justice, Government of India. *The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023)*. The Gazette of India, Extraordinary, Part II, Section 1. 2023 (cit. on p. 23).
- [Obj17] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. formal/17-12-05. Object Management Group. 2017 (cit. on p. 27).
- [OWA21] OWASP Foundation. *OWASP Top 10:2021*. 2021. URL: <https://owasp.org/Top10/> (cit. on p. 16).
- [PM15] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. 8th ed. New York: McGraw-Hill Education, 2015 (cit. on pp. 17, 21).
- [PM99] Niels Provos and David Mazières. “A Future-Adaptable Password Scheme”. In: *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*. 1999, pp. 81–91 (cit. on p. 15).
- [Ram26] Sebastián Ramírez. *FastAPI Documentation*. 2026. URL: <https://fastapi.tiangolo.com/> (cit. on p. 9).

- [Sin84] Roger W. Sinnott. “Virtues of the Haversine”. In: *Sky and Telescope* 68.2 (1984), p. 159 (cit. on p. 14).
- [Som16] Ian Sommerville. *Software Engineering*. 10th ed. Boston: Pearson, 2016 (cit. on pp. 9, 21).
- [SQL26a] SQLAlchemy Authors. *SQLAlchemy 2.0 Documentation*. 2026. URL: <https://docs.sqlalchemy.org/en/20/> (cit. on p. 9).
- [SQL26b] SQLite Consortium. *Appropriate Uses For SQLite*. 2026. URL: <https://www.sqlite.org/whentouse.html> (cit. on pp. 9, 21).
- [Tha26] Thapar Institute of Engineering and Technology. *TIET Report Template (tietreport.cls v1.0.4)*. Overleaf. 2026. URL: <https://www.overleaf.com/read/rjgcdqyqkxnd> (cit. on p. 8).
- [Uni24] United Nations Environment Programme. *Food Waste Index Report 2024*. Technical report. Nairobi: United Nations Environment Programme, 2024 (cit. on p. 5).
- [Ven26] Raghav B. Venkataramaiyer. *UCS503 Semester Project: Deliverables and Evaluation Timeline*. Thapar Institute of Engineering and Technology. 2026. URL: <https://tiet-ucs503.github.io/semester-project/> (cit. on pp. 5, 23).